9 Práce s řetězci

V této kapitole se dozvíte:

- Jakým způsobem je možné vytisknout řetězec v PHP.
- Jak vyhledávat v řetězci.
- Jak upravovat řetězce.
- Jak spojovat a rozdělovat řetězce.

Po jejím prostudování byste měli být schopni:

- Umět vytisknout řetězec a vytvořit formátovaný výstup.
- Umět vyhledávat v řetězci.
- Umět upravovat řetězce pomocí předdefinovaných funkcí.
- Umět rozdělit řetězec na pole a spojit pole do řetězce.

Klíčová slova této kapitoly:

řetězce, echo, print, printf, strlen, strpos, htmlspecialchars, htmlentities, implode, explode, str_split.

Doba potřebná ke studiu: 4 hodiny

Průvodce studiem



Studium této kapitoly není náročné. Jedná se o přehledovou kapitolu, ve které jsou představeny vybrané funkce pro práci s řetězci. Jejich výčet není úplný, jsou představeny pouze některé z nich. V kapitole se postupně seznámíte se způsoby vypisování řetězců, s vyhledáváním v řetězci, s úpravou řetězců a s rozdělováním a spojováním řetězců.

Prakticky každá webová aplikace pracuje s řetězci. Při práci s aplikací je potřeba zpracovávat vstupy od uživatele nebo generovat výstupy. V některých případech se jedná o číselné údaje, ale častěji se jedná o řetězce znaků. Jazyk PHP obsahuje mnoho funkcí pro práci s řetězci, které usnadňují tvorbu aplikací. V této kapitole se seznámíme s nejdůležitějšími z nich.

9.1 Vypsání řetězce



Základní operací s jakýmkoliv řetězcem je jeho vypsání na obrazovku, resp. do okna webového prohlížeče. Doposud jsme se setkali s funkcí `echo`, která vypisuje řetězec. Ve skutečnosti není `echo` funkcí, ale jedná se o konstrukt jazyka PHP. Proto při volání nemusí být použity závorky. Konstrukt `echo` se nechová jako funkce, takže nemůže vždy být použit jako funkce.

Konstrukt `echo` vypisuje jak konstanty (řetězcové nebo číselné), tak obsah proměnných. Pro výpis řetězcových konstant je možné použít jak klasické, tak jednoduché uvozovky. Spojování řetězců nebo obsahu proměnných do jednoho výpisu je možné pomocí operátoru **.** (tečka) nebo **,** (čárka). Pro výpis

speciálních znaků (uvozovky, obrácené lomítko atd.) je potřeba použít escape sekvenci – přidáním obráceného lomítka před speciální znak.

Kromě konstruktu `echo` je možné k výpisu řetězce použít konstrukt `print`. Opět se nejedná o klasickou funkci, ale o jazykový konstrukt, takže při volání nemusí být uvedeny závorky. Hlavní rozdíl mezi konstrukty `echo` a `print` je v tom, že `print` vždy vrací hodnotu `1`. Další rozdíl je v tom, že pokud chceme ve výpisu pomocí konstruktu `print` spojit více řetězců, můžeme použít pouze operátor `.` (tečka).

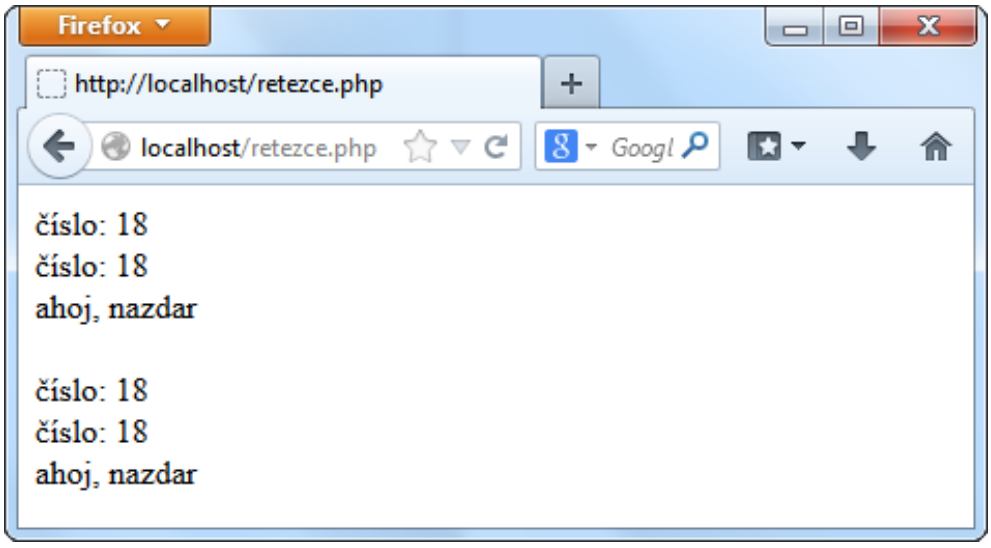
Následující výpis ukazuje použití konstruktů `echo` a `print`. Obrázek pod výpisem pak zobrazuje výstup v prohlížeči.

Výpis 9.1 Ukázka použití konstruktů echo a print.



```
<?php
  $cislo = 18;
  echo "číslo: $cislo<br>";
  echo "číslo: " . $cislo . "<br>";
  echo "ahoj" , ", " , "nazdar<br><br>";

  print "číslo: $cislo<br>";
  print "číslo: " . $cislo . "<br>";
  print "ahoj" . ", " . "nazdar";
?>
```



Obrázek 9.1 Použití konstruktů echo a print.

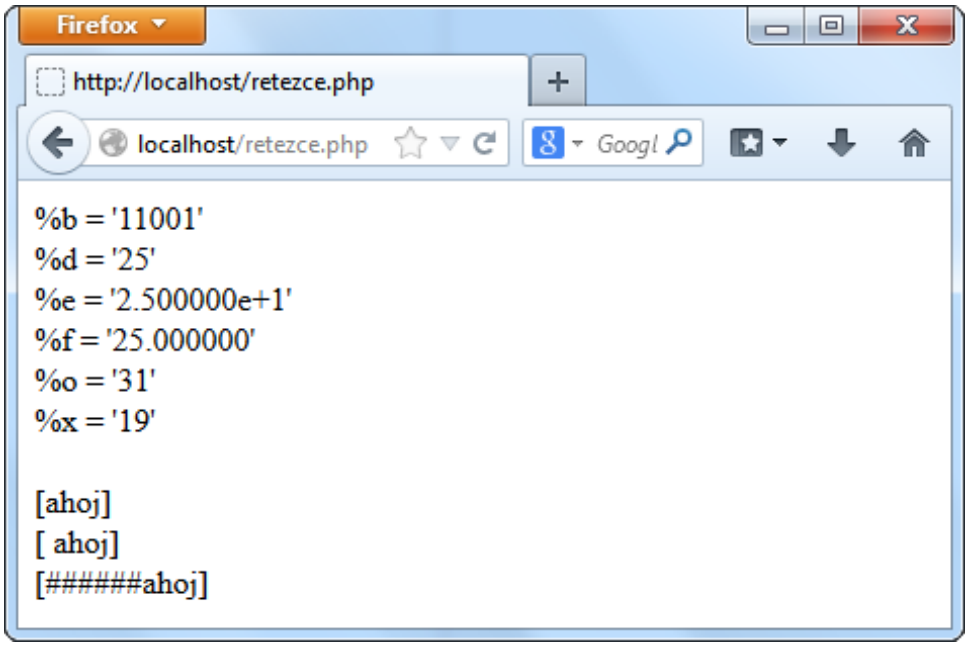
Další funkcí, kterou je možné použít k výpisu řetězců do okna prohlížeče, je funkce `printf()`. Tato funkce slouží k vytvoření formátovaného výstupu a vrací celé číslo – délku vypsaného řetězce. Počet parametrů této funkce není

pevně dán, liší podle toho, jak vypadá vstupní řetězec. Prvním parametrem funkce je vstupní řetězec, který se má vypsat. Tento řetězec může obsahovat formátovací řetězce, které určují, jak se vypíše obsah proměnných. Formátovací řetězce vždy začínají znakem `%` (procento), za kterým následují znaky určující způsob výpisu. Můžeme například určit délku vypsaného řetězce, počet vypisovaných desetinných míst apod. Počet použitých formátovacích řetězců určuje počet zbývajících parametrů funkce `printf()`. Jako další parametry funkce je potřeba uvést názvy proměnných nebo konstanty, na které se mají aplikovat formátovací pravidla. Kompletní popis funkce `printf()` je možné nalézt v dokumentaci k jazyku PHP, zde si ovšem uvedeme několik příkladů.

```
<?php
  $a = 25;
  // binarni reprezentace
  printf("%%b = '%b'<br>", $a);
  // klasicka celociselna reprezentace
  printf("%%d = '%d'<br>", $a);
  // vedecka reprezentace
  printf("%%e = '%e'<br>", $a);
  // reprezentace jako desetinne cislo
  printf("%%f = '%f'<br>", $a);
  // osmickova soustava
  printf("%%o = '%o'<br>", $a);
  // hexadecimalni soustava
  printf("%%x = '%x'<br><br>", $a);

  $b = "ahoj";
  // klasicky vypis
  printf("[%s]<br>", $b);
  // vypis o delce 10 znaku, zarovnani vpravo
  printf("[%10s]<br>", $b);
  // vypis o delce 10 znaku, zarovnani vpravo,
   doplneni znaku # na volna mista
  printf("[%'#10s]<br>", $b);
?>
```





Obrázek 9.2 Ukázka použití funkce printf().

9.2 Procházení řetězců, vyhledávání v řetězcích



Při tvorbě webových aplikací nepotřebujeme řetězce pouze vypisovat, ale potřebujeme s nimi také dále pracovat. V případě, že chceme přistupovat k jednotlivým znakům řetězce, můžeme použít dva způsoby. Oba jsou velmi podobné. S řetězci v jazyce PHP můžeme pracovat jako s poli. Použitím hranatých závorek `[ ]` a uvedením indexu požadovaného znaku získáme jeden znak z řetězce. První znak v řetězci má index `0`, druhý index `1` atd. Druhou možností přístupu k jednotlivým znakům řetězce je použití složených závorek `{ }`. I v tomto případě je v závorkách uveden index požadovaného znaku.

Funkce `strlen()` zjišťuje délku řetězce. Parametrem funkce je řetězec, u kterého chceme zjistit délku (počet znaků). Funkce vrací celé číslo udávají počet znaků řetězce. Pokud je řetězec prázdný, vrátí funkce hodnotu `0`.

Pro zjištění, zda se nějaký řetězec nebo znak nachází v jiném řetězci, slouží funkce `strpos()`. Funkce `strpos()` vrací pozici prvního výskytu hledaného řetězce v jiném řetězci. Pokud není hledaný řetězec nalezen, funkce vrátí hodnotu `false`. Funkce `strpos()` má 2 povinné parametry. Prvním parametrem je řetězec, ve kterém chceme hledat. Druhým parametrem je hledaný řetězec. Třetím, volitelným, parametrem je index, na kterém chceme zahájit hledání. Jako třetí parametr musí být zadáno nezáporné číslo. Pokud není třetí parametr zadán, hledá se od začátku řetězce (od indexu 0). Funkce `strpos()` je case senzitivní, tj. při hledání řetězce se rozlišují velká a malá

písmena. Funkce `stripos()` má stejné parametry i návratové hodnoty jako funkce `strpos()`, ale rozdíl je v tom, že funkce `stripos()` není case senzitivní, tj. při hledání se nerozlišují velká a malá písmena.

Dalšími velmi podobnými funkcemi jsou `strrpos()` a `strripos()`. Mají stejné parametry i návratové hodnoty jako funkce `strpos()`, ale při hledání zjišťují pozici posledního výskytu hledaného řetězce. Funkce `strrpos()` je case senzitivní, funkce `strripos()` je case insenzitivní. Třetí parametr těchto funkcí může být záporné číslo. V případě, že zadáme záporné číslo, odečte se od konce řetězce zadaný počet znaků a hledání probíhá směrem k začátku řetězce. Pokud tedy jako třetí parametr zadáme hodnotu -2, začne hledání na předposledním znaku řetězce.

Následující výpis ukazuje použití výše uvedených funkcí. Obrázek pod výpisem pak zobrazuje výsledek v okně prohlížeče.

Výpis 9.2 Ukázka použití funkcí strlen(), strpos() a strrpos().

```
<?php
  $text = "Dobrý den! Jak se máte?";
  echo "Třetí znak: " . $text[1] . "<br>";
  echo "První znak: " . $text{0} . "<br>";
  echo "Počet znaků: " . strlen($text) . "<br>";

  // case senzitivni hledani, retezec DEN neni
   nalezen
  $pozice = strpos($text, "DEN");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'DEN': $pozice<br>";

  // case insenzitivni hledani, retezec DEN je
   nalezen
  $pozice = stripos($text, "DEN");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'DEN': $pozice<br>";

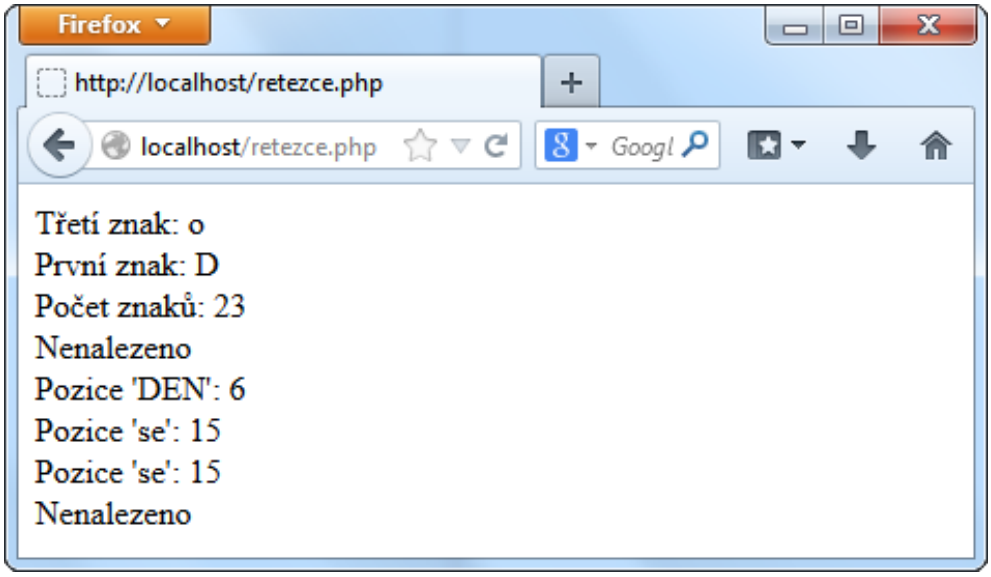
  // hledani od indexu 10 (od 11. znaku)
  $pozice = stripos($text, "se", 10);
  if ($pozice === false)
    echo "Nenalezeno<br>";
```



```
  else
    echo "Pozice 'se': $pozice<br>";

  // hledani posledniho vyskytu
  $pozice = strrpos($text, "se");
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'se': $pozice<br>";

  // hledani od konce retezce smerem k zacatku
   retezce
  $pozice = strrpos($text, "se", -10);
  if ($pozice === false)
    echo "Nenalezeno<br>";
  else
    echo "Pozice 'se': $pozice<br>";
?>
```



Obrázek 9.3 Použití funkcí strlen(), strpos() a strrpos().

9.3 Úprava řetězců



Další operací, kterou můžeme provádět s řetězci, je jejich úprava. Pod pojmem úprava řetězce si můžeme představit mnoho různých činností. Jako první si uvedeme situaci, kdy chceme některé znaky v řetězci nahradit jinými znaky. K nahrazení částí řetězce jiným řetězcem slouží funkce `str_replace()`. Tato funkce má tři povinné parametry. Prvním parametrem je řetězec, který bude v původním řetězci nahrazen jiným řetězcem. Druhým parametrem je řetězec, který nahradí řetězec uvedený jako první parametr. Třetím parametrem je řetězec, ve kterém se úpravy provádějí. Změny se provedou ve všech

výskytech nahrazovaného řetězce. Funkce vrací nový řetězec po úpravách. Na pozici všech třech povinných parametrů se může objevit také pole. Funkce může pole i vracet. Funkce má ještě volitelný čtvrtý parametr, který se předává odkazem. Pokud je čtvrtý parametr uveden, je do něj uložen počet provedených úprav. Funkce `str_replace()` je case senzitivní. Jazyk PHP obsahuje i case insenzitivní variantu s názvem `str_ireplace()`, která má stejné parametry jako funkce `str_replace()`.

Další dvě funkce sloužící pro úpravu řetězců v PHP souvisí s jazykem HTML, který se používá pro tvorbu webových stránek. Při zpracování vstupů od uživatele je většinou potřeba, aby byla zadaná data ve formě prostého textu. Pokud uživatel vyplňuje nějaká dat např. do prvku typu `textarea`, může samozřejmě zadat i HTML značky. Zadání HTML značek může být použito například k útokům na webovou aplikaci. Funkce `htmlspecialchars()` dokáže nahrazuje znaky jazyka HTML (konkrétně jde o znaky `&`, `"`, `'`, < a >) přepsat HTML entitami. Například znak `&` je nahrazen entitou `&`, znaky < a > jsou nahrazeny entitami `<` a `>`. Funkce má jeden povinný parametr, kterým je vstupní řetězec. Funkce vrací upravený řetězec. Pokud při převodu nastane nějaká chyba, funkce vrátí prázdný řetězec. Funkce má i další volitelné parametry, které souvisí s nastavením znakové sady řetězce apod. Podobnou funkcí je funkce `htmlentities()`, ale na rozdíl od předchozí funkce nahrazuje všechny znaky, pro které existuje HTML entita. Parametry i návratová hodnota funkce jsou stejné jako u funkce `htmlspecialchars()`. Opačnými funkcemi k funkcím `htmlspecialchars()` a `htmlentities()` jsou funkce `htmlspecialchars_decode()` a `html_entity_decode()`, které převádějí HTML entity na odpovídající znaky.

Při zpracování víceřádkového textu (zadaného například do prvku `textarea`) je někdy potřeba zachovat rozložení textu na řádky při zobrazení na HTML stránce. V prvku `textarea` je přechod na nový řádek proveden pomocí znaku pro nový řádek (např. `\n`), v kódu HTML je potřeba použít značku `<br>` (případně `<br/>` pro kód XHTML). K nahrazení znaků nového řádku HTML značkami slouží funkce `nl2br()`, jejímž parametrem je řetězec, ve kterém chceme provést změnu. Kromě toho má funkce druhý, nepovinný, parametr – logickou hodnotu určující, zda se jedná o XHTML nebo o HTML. Hodnota `true` znamená XHTML a je použita jako výchozí hodnota v případě neuvedení druhé parametru.

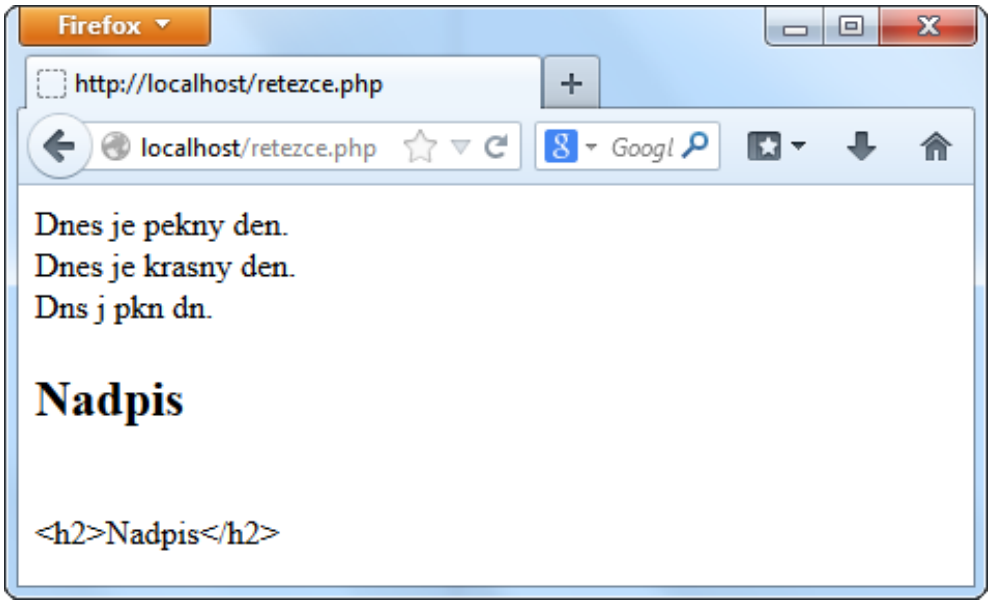
Funkce `wordwrap()` slouží k vložení HTML značky pro konec řádku po zadaném počtu znaků. Ve výchozím nastavení je maximální délka řádku `75` znaků a značka `<br>` se vždy vloží na pozici mezery. Pomocí volitelných parametrů lze nastavit kratší délku řádku i to, že se HTML značka vloží do slova po dosažení zadaného počtu znaků (může se projevit v případě velmi dlouhých slov).

Posledními funkcemi, které zde uvedeme, jsou funkce `stripslashes()` a `addslashes()`. Funkce `addslashes()` přidá před speciální znaky (jednoduché i dvojité uvozovky, obrácené lomítko a nulový bajt) znak obráceného lomítka. Tím pádem už je při zpracování skriptu považován za obyčejný znak. Při zpracování dat zaslaných metodou GET nebo POST nebo pomocí cookie se tato metoda volá automaticky. Předpokladem je, že v konfiguraci PHP je zapnutá volba Magic Quotes. Funkce `stripslashes()` má opačnou funkčnost jako funkce `addslashes()`. Ze vstupního řetězce odstraňuje znak obráceného lomítka.

Výpis 9.3 Ukázka použití funkcí str_replace() a htmlspecialchars().



```
<?php
  $text = "Dnes je pekny den.";
  echo $text . "<br>";
  echo str_replace("pekny", "krasny", $text) .
   "<br>";
  $samohlasky = array("a", "e", "i", "o", "u", "y");
  echo str_ireplace($samohlasky, "", $text) . "<br>";
  $text = "<h2>Nadpis</h2>";
  echo $text . "<br>";
  echo htmlspecialchars($text) . "<br>";
?>
```



Obrázek 9.4 Použití funkcí str_replace() a htmlspecialchars().

9.4 Rozdělování a spojování řetězců



V některých situacích je potřeba spojit prvky pole do jednoho řetězce. K tomuto účelu je v PHP k dispozici funkce `implode()`. Funkce je k dispozici ve dvou variantách. První varianta má dva parametry – prvním parametrem je řetězec, který se při spojování vloží mezi jednotlivé prvky pole. Druhým parametrem je pole, jehož prvky chceme spojit. Druhá varianta funkce má pouze jeden parametr, kterým je pole, jehož prvky chceme spojit. Mezi jednotlivé prvky pak není vložen žádný oddělovací řetězec. Obě varianty funkce `implode()` vrací výsledný řetězec. Alternativou je použití funkce `join()`, která je ale pouze aliasem funkce `implode()` – má tedy stejné parametry i návratovou hodnotu.

Opačnou operací je rozdělení řetězce na části, které se uloží do pole. K tomuto účelu je možné použít funkci `explode()`, která má dva povinné parametry. Prvním parametrem je řetězec představující oddělovač, podle kterého se vstupní řetězec rozdělí. Druhým parametrem je řetězec, který chceme dělit. Kromě toho je možné použít třetí volitelný parametr, kterým je umožňuje nastavit výsledný počet prvků pole. Návratovou hodnotou je pole řetězců.

V případě, že vstupní řetězec nechceme dělit podle nějakého oddělovače, ale chceme ho rozdělit na řetězce stejné délky, můžeme použít funkci `str_split()`, která má jako povinný parametr vstupní řetězec. Maximální délka jednotlivých řetězců se nastavuje pomocí druhého volitelného parametru.

Pokud není druhý parametr uveden, rozdělí se vstupní řetězec po jednotlivých znacích.

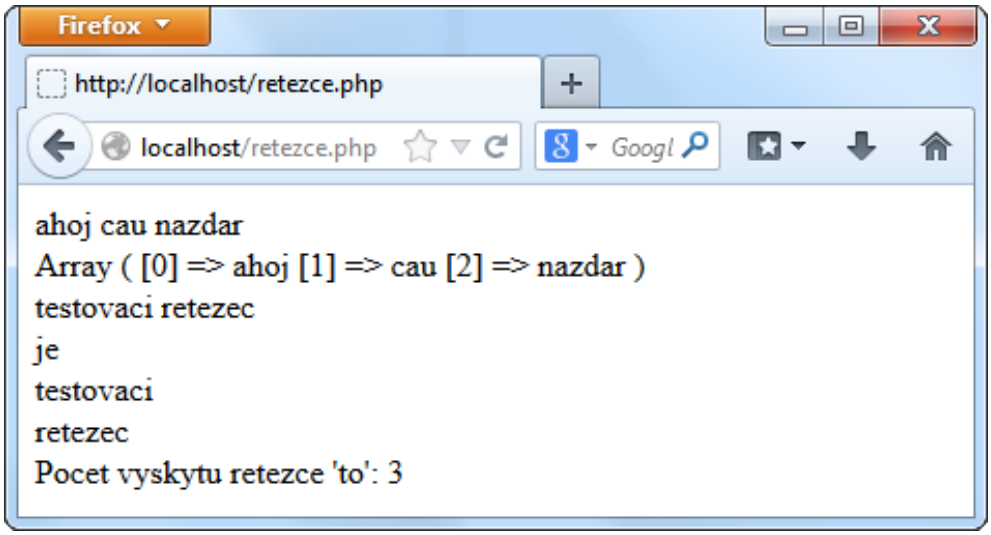
Jazyk PHP obsahuje i funkci `substr()`, která vrací část vstupního řetězce. Návratovou hodnotou je také řetězec. Funkce má dva povinné parametry, kdy prvním parametrem je vstupní řetězec a druhým je index znaku, na kterém má začínat podřetězec. První znak má index `0`. Třetím parametrem je délka podřetězce. Tento parametr je volitelný. Pokud není zadán, vrátí se podřetězec od zadané pozice do konce vstupního řetězce.

Funkce `substr_count()` slouží k určení, kolikrát se ve vstupním řetězci vyskytuje určitá část. Funkce vrací celé číslo. Povinnými parametry jsou vstupní řetězec a řetězec, jehož výskyt se v něm vyhledává. Volitelnými parametry je možné nastavit, kde začne a skončí hledání ve vstupním řetězci.

Výpis 9.4 Použití funkcí implode(), explode(), substr() a substr_count().



```
<?php
  $slova1 = array("ahoj", "cau", "nazdar");
  $text = implode(" ", $slova1);
  echo "$text<br>";
  $slova2 = explode(" ", $text);
  print_r($slova2);
  echo "<br>";
  $text = "toto je testovaci retezec";
  echo substr($text, 8) . "<br>";
  echo substr($text, 5, 2) . "<br>";
  echo substr($text, 8, -8) . "<br>";
  echo substr($text, -7) . "<br>";
  echo "Pocet vyskytu retezce 'to': " .
   substr_count($text, "to");
?>
```



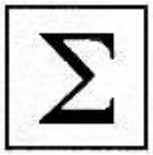
Obrázek 9.5 Použití funkcí implode(), explode(), substr() a substr_count().

Kontrolní otázky:



1. Jakým způsobem je možné vytisknout řetězec?
2. Jakou funkcí je možné vytvořit formátovaný řetězec?
3. Jaký je rozdíl mezi funkcemi `implode()` a `join()`?
4. K čemu slouží funkce `substr_count()`?

Shrnutí obsahu kapitoly



V této kapitole jste s předdefinovanými funkcemi v PHP, které slouží pro práci s textovými řetězci. Tisk řetězce je možné provést příkazy echo a print. Pro vytvoření formátovaného výstupu slouží funkce printf. K řetězci lze přistupovat jako k poli, pro práci s jednotlivými znaky se používají index uvedené v hranatých nebo složených závorkách. Pomocí funkcí lze v PHP zjistit délku řetězce a najít první nebo poslední výskyt řetězce v jiném řetězci. Úpravu řetězce je možné provést nahrazením části řetězce jiným řetězcem, přidáním obráceného lomítka před speciální znaky nebo nahrazením speciálních znaků jazyka HTML odpovídajícími HTML entitami. Řetězce lze také rozdělit podle zadaného oddělovače na pole dílčích řetězců a nebo je možné prvky pole spojit do jednoho řetězce. Jazyk PHP obsahuje mnoho funkcí, jejich popis v této kapitole není úplný, jsou zmíněny jen některé z funkcí.